

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Announcements

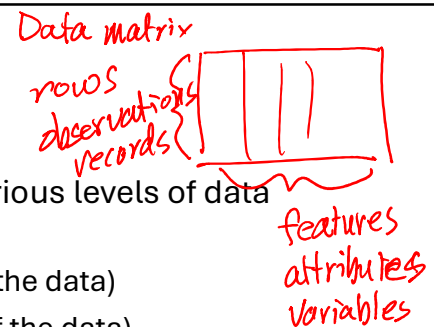
- Notebook files on Lamaku have been updated.

Lecture 3

- Exploratory Data Analysis
 - Inspecting data
 - Summary statistics
 - Missing values
 - Sorting
 - Basic visualization with Pandas

Exploring your data

- Data analysis begins with exploring the data. Various levels of data exploration can be performed. For example,
 - How many records are there? (high-level view of the data)
 - How many columns are there? (high-level view of the data)
 - What is a column's average and standard deviation? (intermediate-level view of the data)
 - What are the number of unique values in a column? How many of each are there? (intermediate-level view of the data)
 - What are the distributions of the data in various columns? (low-level view of the data)
 - etc.
- We will explore Pandas functions to cover interesting use cases.



Pandas functionality

- DataFrames have a number of useful features implemented as:
 - Attributes that pertain to features (constants of the data)
 - Number of rows, type of a given column x, etc.
 - Methods to act on the data
 - Non-transformative functions, for example:
 - Give me the maximum value in column x.
 - Transformative (but not in place) functions, for example:
 - Remove column x, sort by column x, filter values that are less than some value in column x.
 - Functions to apply some custom functionality (your own function) to the data. For example:
 - Normalize the values in a column x by subtracting the mean of the column and dividing by the standard deviation.

Attributes

- If you have loaded a table, you probably want to see how the data is structured
- The following three attributes are useful to get a sense of the structure of your data:

Attribute	Description
shape	Return a <code>tuple</code> representing the dimensionality of the DataFrame (r, c)
size	Return an <code>int</code> representing the number of elements in this object r x c
dtypes	Return the data types in the DataFrame

- Recall that the syntax for accessing a DataFrame's attribute is:

`ObjectName.attributeName`

- Example: `spending_df.shape`

```

spending_df = pd.read_csv("data/spending_10k.csv", index_col='unique_id')
spending_df.head()

```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
NX531425	1255626040	FAMILY PRACTICE	METFORMIN HCL	30	135.24
QG879256	1699761833	FAMILY PRACTICE	ALLOPURINOL	30	715.76
FW363228	1538148804	INTERNAL MEDICINE	LOSARTAN POTASSIUM	146	1056.47
WD733417	1730200619	PSYCHIATRY	OLANZAPINE	13	28226.97
XW149832	1023116894	FAMILY PRACTICE	PRAVASTATIN SODIUM	348	8199.48

```
spending_df.shape
```

```
(10000, 5)
```

```
spending_df.size
```

```
50000
```

```
spending_df.dtypes
```

```

doctor_id      int64
specialty      object
medication      object
nb_beneficiaries int64
spending      float64
dtype: object

```

Inspecting data types

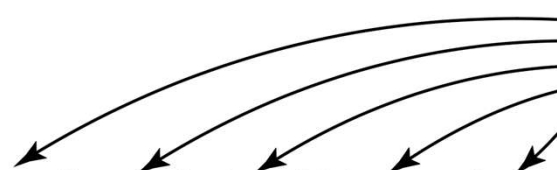
- DataFrame columns contain data, whose type is inferred by Pandas.
- Some operations in Pandas are only valid on certain types. For example:
 - Computing average on spending column is valid.
 - Computing the median on the medication column is invalid.
- Inferring doctor_id as a number that can be averaged is perhaps wrong.
 - Perhaps should be set as a string (text) instead.

Inspecting data types (cont'd.)

- dtypes field holds the data type of each column of the DataFrame.
- dtype field holds the data type for a Series.
 - Holds the data type in that column.

```
In [78]: spending_df.dtypes
Out[78]:
```

```
doctor_id      int64
specialty      object
medication      object
nb_beneficiaries  int64
spending      float64
dtype: object
```



unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
AB789982	1952310666	Psychiatry	CLONAZEPAM	226	1,848.88
AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
GH890091	1346358827	Family	HYDROCODONE	331	8,511.14
YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	1,964.49
....					

Methods

- Python methods are functions that are accessible via an object instance.
 - Apply to the object that calls them.
- Syntax:
 - `ObjectName.methodName()`

Methods (cont'd.)

- DataFrame have many built-in methods to summarize our data. Ex:

Method	Description
<code>head()</code>	Return first n rows
<code>tail()</code>	Return last n rows
<code>min()</code> , <code>max()</code>	Computes the numeric or alphanumeric minimum and maximum values
<code>sum()</code> , <code>mean()</code> , <code>std()</code> , <code>var()</code>	Computes the column-wise sum, mean, standard deviation and variance in a <code>Series</code> or a <code>DataFrame</code>
<code>count()</code>	Returns the number of non-missing values in a <code>Series</code> or a <code>DataFrame</code>
<code>value_counts()</code>	Returns the frequency for each value in a <code>Series</code> → series method
<code>describe()</code>	Computes column-wise summary statistics

- There are numerous other functions to explore the data. Those will be covered throughout the semester.

```
med = spending_df["medication"]

# recall that shape returns a tuple
med.shape
```

```
(10000,)
```

```
med = spending_df["medication"]
med.count()
```

```
10000
```

```
# frequency of occurrence
```

```
med.value_counts()
```

```
medication
LEVOTHYROXINE SODIUM    150
GABAPENTIN              126
POTASSIUM CHLORIDE      125
LISINOPRIL              118
METFORMIN HCL           118
...
CEFAZOLIN SODIUM        1
OXAZEPAM                 1
LANTHANUM CARBONATE      1
LIDOCAINE HCL/PF         1
ETHOTOIN                 1
Name: count, Length: 617, dtype: int64
```

DataFrame Axes

- Computing statistics on the data is a critical aspect of exploratory data analysis.
- Provides valuable information about the data.
 - For example, what medications are most prescribed, which doctors or specialty produce most prescriptions, what is the total of opiate prescriptions per state, etc.

DataFrame Axes (cont'd.)

- We normally think of computations on columns. What if you want to perform a computation row-wise?
 - Having another set of functions would be redundant and a bad idea.
 - Bad idea: `compute_avg_row()` and `compute_avg_col()`, `compute_std_row()` and `compute_std_col()`, etc.
- Instead of duplicating functionality, Python uses the concept of **axis**.

DataFrame Axes (cont'd.)

- For instance, the methods `sum()`, `min()`, `max()`, etc. can all be applied row- or column-wise.
- Note that they have a parameter called `axis`.
 - Default is 0, which means the operation is applied column-wise (i.e., along the rows).
 - This concepts may be a bit confusing but you will get ample opportunities to explore it.

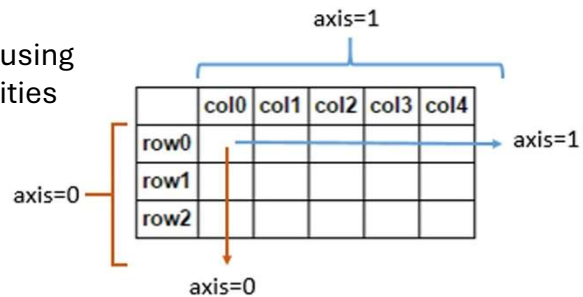


Image from:

<https://stackoverflow.com/questions/39283339/what-is-the-meaning-of-axis-parameter-in-a-pandas-dataframe>

`axis='rows'`
or
`axis=0`

axis = 'index'

	Col_A	Col_B	Col_C
row_0	1	2	3
row_1	2	3	4
row_2	3	4	5
row_3	4	5	6
row_4	5	6	7
row_5	6	7	8
	SUM	SUM	SUM
	21	27	33

`axis='columns'`
or
`axis=1`

	Col_A	Col_B	Col_C	
row_0	1	2	3	SUM → 6
row_1	2	3	4	SUM → 9
row_2	3	4	5	SUM → 12
row_3	4	5	6	SUM → 15
row_4	5	6	7	SUM → 18
row_5	6	7	8	SUM → 21

```
spending_df.sum()
```

'str1' + 'str2' = 'str1str2'

```
doctor_id          15037655350169
specialty          FAMILY PRACTICEFAMILY PRACTICEINTERNAL MEDICIN...
medication          METFORMIN HCLALLOPURINOLLOSARTAN POTASSIUMOLAN...
nb_beneficiaries    509183
spending            43338385.95
dtype: object
```

```
spending_df.sum?
```

Signature:

```
spending_df.sum(
    axis: 'Axis | None' = 0,
    skipna: 'bool' = True,
    numeric_only: 'bool' = False,
    min_count: 'int' = 0,
    **kwargs,
)
```

Docstring:

Return the sum of the values over the requested axis.

⋮

Descriptive statistics

- We'll explore some more functions on simple, manually created DataFrame.
- Making a test dataset is always useful when not sure how something works after reading the documentation.

```
df = pd.DataFrame({'A': [0, 1, 2], 'B': [5, 6, 7]})
df
```

	A	B
0	0	5
1	1	6
2	2	7

Descriptive statistics (cont'd.)

- As discussed, the method `mean()` work column-wise by default.
- You will see that the result is a Series.

```
print('Column-wise mean:')  
print(df.mean())
```

Column-wise mean:

A 1.0

B 6.0

dtype: float64

- To compute the row-wise mean, we need to change the `axis` to 1.

```
print('Row-wise mean:')  
print(df.mean(axis=1))
```

Row-wise mean:

0 2.5

1 3.5

2 4.5

dtype: float64

Descriptive statistics (cont'd.)

- Descriptive statistics can also be applied to a Series object.
 - If you are only interested in a subset of the rows or columns of the DataFrame.

```
print('Series mean:')  
print(df['A'].mean())
```

Series mean:

1.0

Descriptive statistics: medical spending data

- `spending_df` contains both numeric and non-numeric columns.
 - Calling methods such as `mean()` on the full DataFrame fails due to non-numeric columns (e.g., medication).
 - Use `numeric_only=True` to compute averages only on numeric columns.

```
spending_df.mean(numeric_only=True)
```

```
doctor_id      1.503766e+09
nb_beneficiaries 5.091830e+01
spending        4.333839e+03
dtype: float64
```

Missing values

- In actual data, missing values in a table can occur under different forms:
 - Blank spaces
 - Empty strings `""`
 - Some predetermine identifiers: E.g., MISSING, NOVAL, etc. N/A
- These values can often skew or “break the analysis.”
 - NOVAL in a column with number results in the whole column being converted to object.
- Custom handling of the missing values is often necessary.
 - Often involved turning those values into Pandas’ NaN equivalent.
- In Pandas, `NaN` represents a “Not a Number”, i.e., a lack of value.
 - NaN represents undefined or “unrepresentable” value.
 - It is not 0, an empty string, or the Boolean False.

```

np.nan, type(np.nan)

(nan, float)

np.nan == 0, np.nan < 0, np.nan > 0, np.nan == "", np.nan == False, np.nan == None

(False, False, False, False, False, False)

np.nan == np.nan

False

equality

### NumPy, for efficiency reasons, uses the same object in memory for its NaN values.
### The 'is' operator in Python checks for object identity,
### i.e., whether they are the same object in memory.

np.nan is np.nan

True

pd.isna(np.nan), pd.isnull(np.nan), np.isnan(np.nan)

(True, True, np.True_)

```

Counting missing values

- How do summary operations handle empty (missing) values?
- As discussed, we explore using a small dataset.

```

na_df = pd.DataFrame({"Col A": [np.nan, np.nan], "Col B": [np.nan, np.nan]})
na_df

```

	Col A	Col B
0	NaN	NaN
1	NaN	NaN

```
na_df.shape
```

```
(2, 2)
```

```
na_df.size
```

Counting missing values (cont'd.)

- In the example, Pandas reports that the DataFrame contains 2 rows and 2 columns with a total of 4 values.
 - We know that there are actually no real data in this DataFrame.
- Rather than using `shape` and `size`, we can instead use the built-in `count()` method.
 - It automatically excludes missing values.
- An easy way to assess the level of missing values in your data.

```
na_df.count()
```

```
Col A    0
Col B    0
dtype: int64
```

Counting missing values in columns

- We'll use another example, `spending_missing_values.csv`, which was specifically constructed to have missing values.

```
spending_missing_values_df = pd.read_csv('data/spending_missing_values.csv',
                                         index_col='unique_id',
                                         na_values=['Null'])
spending_missing_values_df.head()
```

'Null'

N/A, 'null'

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	5409.29
TR390895	1639597115	NaN	LOSARTAN POTASSIUM	11.0	65.62
JA436080	1073781571	NaN	LAMOTRIGINE	12.0	8873.70

```

spending_missing_values_df.iloc[4]

doctor_id      1073781571
specialty      NaN
medication     LAMOTRIGINE
nb_beneficiaries  12.0
spending       8873.7
Name: JA436080, dtype: object

spending_missing_values_df.iloc[4, 1], type(spending_missing_values_df.iloc[4, 1])

(nan, float)

x = pd.read_csv('data/spending_missing_values.csv', index_col='unique_id')
x.iloc[4]

doctor_id      1073781571
specialty      Null
medication     LAMOTRIGINE
nb_beneficiaries  12.0
spending       8873.7
Name: JA436080, dtype: object

x.iloc[4, 1], type(x.iloc[4, 1])

('Null', str)

```

Counting missing values in columns (cont'd.)

- The method `count()` counts values by default column-wise.

- The method has the optional axis parameter with a default of `axis=0`, or equivalently `axis='rows'` or `axis='index'`.

- This concept is prevalent when working with Pandas. So please remember the meaning of `axis=0` and `axis=1`.

```

print(spending_missing_values_df.shape)
print("-----")
spending_missing_values_df.count()

```

(54, 5) *df.shape[0]*

```

doctor_id      54
specialty      48
medication     53
nb_beneficiaries  50
spending       50
dtype: int64

```

Counting missing values in columns (cont'd.)

- In the example, columns contain variable number of missing values.
- Using `count(axis=1)` counts the number of non-missing values per row.
- **Question:** In what scenarios might counting valid (or missing) values per row be useful?

The `describe()` method

- The `describe()` summarizes the values in columns of a `DataFrame`.
 - By default, only numerical values are considered.
 - Behavior can be modified by setting the `include` parameter to 'all'.

```
spending_df.describe()
```

	doctor_id	nb_beneficiaries	spending
count	1.000000e+04	10000.000000	10000.000000
mean	1.503766e+09	50.918300	4333.838595
std	2.874269e+08	86.493443	21915.925814
min	1.003010e+09	11.000000	15.020000
25%	1.255580e+09	15.000000	253.237500
50%	1.508818e+09	24.000000	677.970000
75%	1.750467e+09	50.000000	2442.955000
max	1.992999e+09	1987.000000	892027.000000

```
spending_df.describe(include='all')
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
count	1.000000e+04	10000	10000	10000.000000	10000.000000
unique	NaN	75	617	NaN	NaN
top	NaN	INTERNAL MEDICINE	LEVOTHYROXINE SODIUM	NaN	NaN
freq	NaN	3060	150	NaN	NaN
mean	1.503766e+09	NaN	NaN	50.918300	4333.838595
std	2.874269e+08	NaN	NaN	86.493443	21915.925814
min	1.003010e+09	NaN	NaN	11.000000	15.020000
25%	1.255580e+09	NaN	NaN	15.000000	253.237500
50%	1.508818e+09	NaN	NaN	24.000000	677.970000
75%	1.750467e+09	NaN	NaN	50.000000	2442.955000
max	1.992999e+09	NaN	NaN	1987.000000	892027.000000

The info() method

- We can also get a summary of a DataFrame by using the info() method.

```
spending_df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 10000 entries, NX531425 to WH309636
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   doctor_id             10000 non-null   int64
1   specialty             10000 non-null   object
2   medication            10000 non-null   object
3   nb_beneficiaries      10000 non-null   int64
4   spending              10000 non-null   float64
dtypes: float64(1), int64(2), object(2)
memory usage: 468.8+ KB
```

Sorting

- Ordering data is often critical during exploration.
- Examples:
 - Detecting anomalies.
 - Identifying scales and ranges of data (min and max transactions).
 - Identifying important categories.

Sorting (cont'd.)

- We can sort either:
 - Lexicographically by row index `sort_index()`.
 - Since there is only one index, the method does not have any required parameters.
 - Lexicographically on a column value `sort_values()`.
 - Since a DataFrame may have many columns, you need to explicitly specify the column(s) you want to sort on.
- Sorting is by default in ascending order.
 - Behavior can be modified using `ascending` parameter.

```
spending_df.sort_index()
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
AA137628	1619994506	INTERNAL MEDICINE	PRAVASTATIN SODIUM	17	722.11
AA150698	1124243563	CARDIOLOGY	FENOFIBRATE NANOCRYSTALLIZED	29	1327.06
AA256462	1780610477	INTERNAL MEDICINE	FINASTERIDE	81	1508.39
AA309915	1760896161	FAMILY PRACTICE	PRAVASTATIN SODIUM	11	241.33
AA316473	1023104098	INTERNAL MEDICINE	WARFARIN SODIUM	46	529.07
...	...	...	...	...	...
ZZ769373	1245660554	NURSE PRACTITIONER	LISINOPRIL	18	118.82
ZZ814526	1982960043	PHYSICIAN ASSISTANT	METOPROLOL TARTRATE	13	69.52
ZZ861557	1457358053	CARDIOLOGY	WARFARIN SODIUM	26	254.48
ZZ881671	1346689742	INTERNAL MEDICINE	FUROSEMIDE	11	26.08
ZZ934824	1811995723	FAMILY PRACTICE	FENOFIBRATE	30	1987.55

10000 rows × 5 columns

```
spending_df.sort_values('spending', ascending=False)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
RU995312	1467442392	PULMONARY DISEASE	AMBRISENTAN	118	892027.00
SJ341179	1780895854	GASTROENTEROLOGY	LEDIPASVIR/SOFOSBUVIR	27	855440.65
HG989315	1992757181	DIAGNOSTIC RADIOLOGY	TERIPARATIDE	285	564901.05
ZE579249	1457375321	NEUROLOGY	GLATIRAMER ACETATE	89	564109.04
UF412652	1891767307	PEDIATRIC MEDICINE	GLYCEROL PHENYLBUTYRATE	12	480637.52
...	...	...	...	...	...
OD445990	1841366820	INTERNAL MEDICINE	FUROSEMIDE	13	19.90
TX420809	1801032297	ADDICTION MEDICINE	LORAZEPAM	14	19.56
MR117563	1629180708	PHYSICIAN ASSISTANT	SIMVASTATIN	11	16.72
YB781831	1801080890	RHEUMATOLOGY	LISINOPRIL	11	15.84
XS313828	1508965476	GASTROENTEROLOGY	ALPRAZOLAM	13	15.02

10000 rows × 5 columns

Method chaining

- Successive application of methods onto an object can be chained and executed in a single line of code.
- The syntax for method chaining is the following:

```
object.method_1().method_2()...
```

- We'll demonstrate using the `sample()` method we haven't covered but that are fairly intuitive.
 - `sample()`: samples (selects randomly) a given number of entries

```

spending_sorted_df = spending_df.sort_values(by="spending", ascending=False)
spending_sorted_top_10_df = spending_sorted_df.head(10)

```

```
spending_sorted_top_10_df
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
RU995312	1467442392	PULMONARY DISEASE	AMBRISENTAN	118	892027.00
SJ341179	1780895854	GASTROENTEROLOGY	LEDIPASVIR/SOFOSBUVIR	27	855440.65
HG989315	1992757181	DIAGNOSTIC RADIOLOGY	TERIPARATIDE	285	564901.05
ZE579249	1457375321	NEUROLOGY	GLATIRAMER ACETATE	89	564109.04
UF412652	1891767307	PEDIATRIC MEDICINE	GLYCEROL PHENYLBUTYRATE	12	480637.52
HI666266	1144277294	NEPHROLOGY	SEVELAMER CARBONATE	242	446874.49
JM425012	1851423701	GASTROENTEROLOGY	LEDIPASVIR/SOFOSBUVIR	14	445448.40
RE720840	1801196829	NURSE PRACTITIONER	CRIZOTINIB	29	385275.65
GJ496150	1851379762	NURSE PRACTITIONER	SEVELAMER CARBONATE	218	343489.69
HY126030	1104998566	NURSE PRACTITIONER	TERIFLUNOMIDE	33	308461.71

```
spending_sorted_top_10_df.sample(2)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
HY126030	1104998566	NURSE PRACTITIONER	TERIFLUNOMIDE	33	308461.71
ZE579249	1457375321	NEUROLOGY	GLATIRAMER ACETATE	89	564109.04

```
spending_df.sort_values(by="spending", ascending=False).head(10).sample(2)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
JM425012	1851423701	GASTROENTEROLOGY	LEDIPASVIR/SOFOSBUVIR	14	445448.40
SJ341179	1780895854	GASTROENTEROLOGY	LEDIPASVIR/SOFOSBUVIR	27	855440.65

Chaining methods syntax

- Chained methods are often written across multiple lines to facilitate reading.
- Python requires a specific syntax to break chained methods across multiple lines.
 - For example, the following generates an error as Python expects instruction to end at the end of the line.

```
spending_df.  
    sort_values(by="spending", ascending=False).  
    head(10).  
    sample(2)
```

```
spending_df.  
    sort_values(by="spending", ascending=False).  
    head(10).  
    sample(2)
```

Cell In[69], line 1

```
spending_df.
```

SyntaxError: invalid syntax

```
(spending_df.  
    sort_values(by="spending", ascending=False).  
    head(10).  
    sample(2))
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
GJ496150	1851379762	NURSE PRACTITIONER	SEVELAMER CARBONATE	218	343489.69
ZE579249	1457375321	NEUROLOGY	GLATIRAMER ACETATE	89	564109.04

(Some) Data visualization

- There is more than enough content to justify semester-long courses on the topic.
 - There is, in fact, a semester-long course on the topic!
- Pandas provides elementary functionality to produce basic plots.
 - Ideal for primary data exploration for getting to know your data.
 - As opposed to formulating or confirming a hypothesis.

The DataFrame plots

- Pandas has methods for various types of plots:

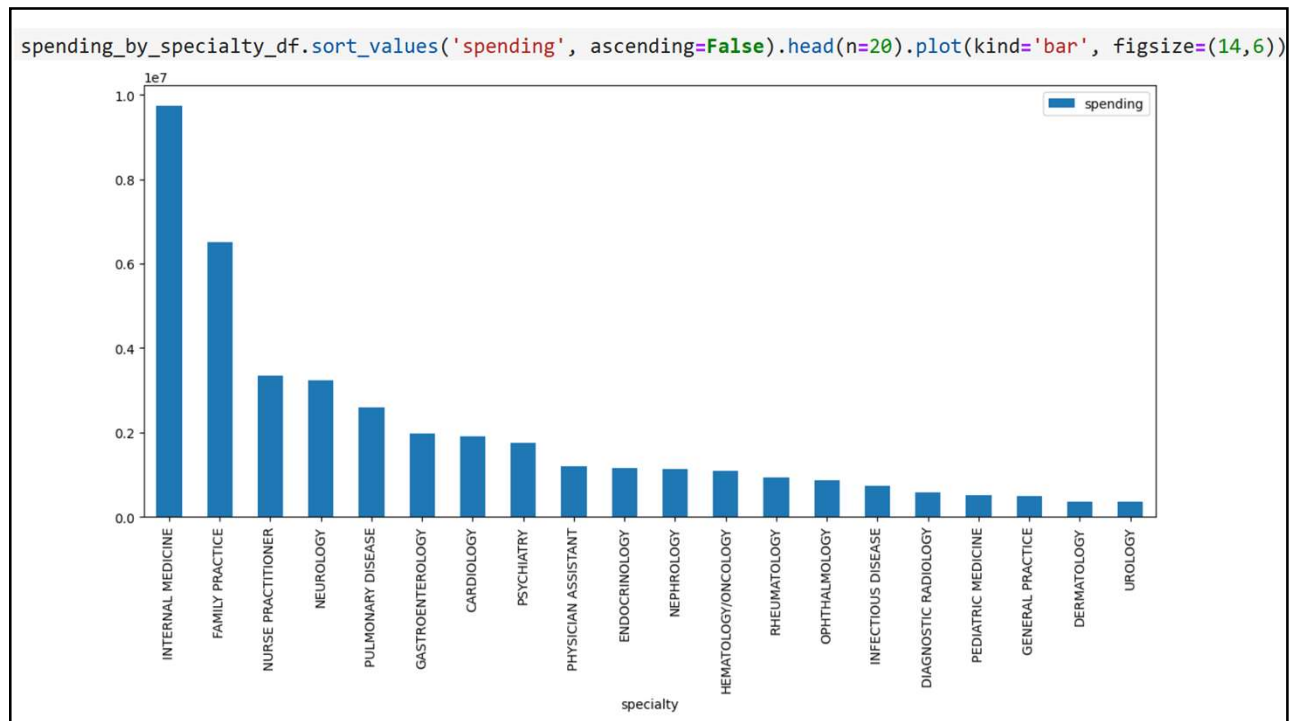
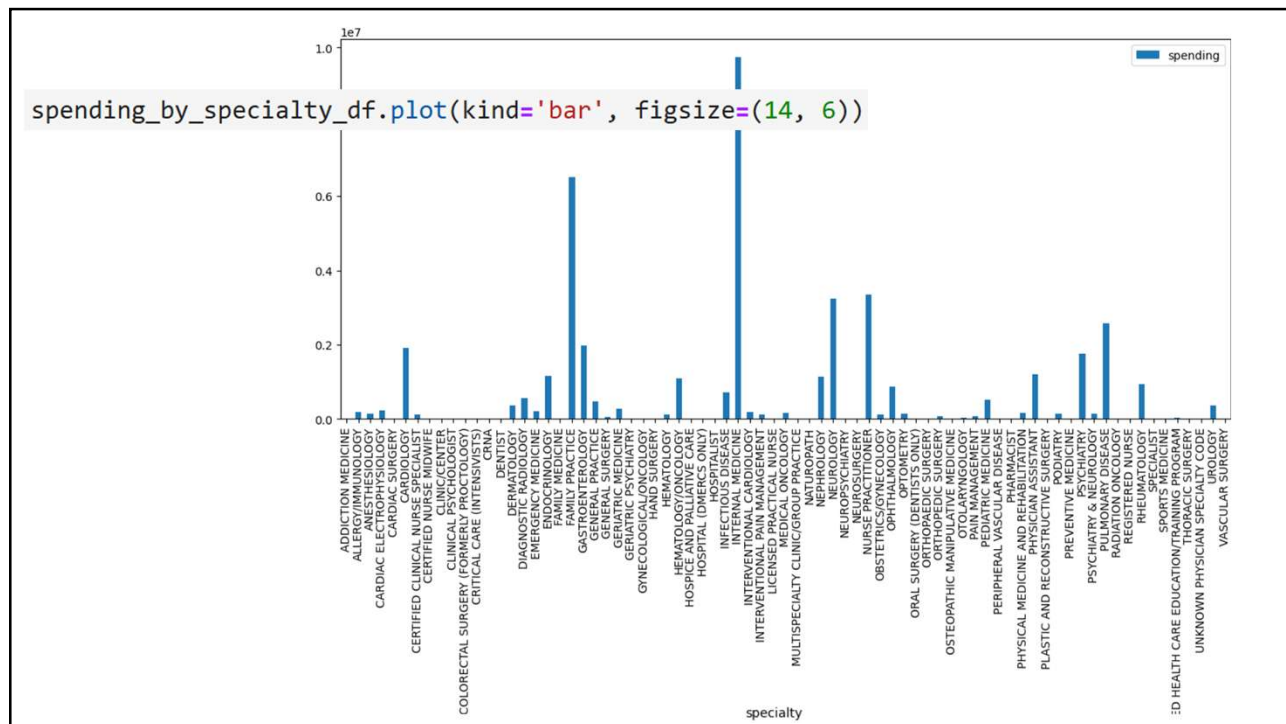
Kind	Description
line	Line plot (default)
bar	Vertical bar plot
barh	Horizontal bar plot
hist	Histogram
box	Boxplot
'density' = kde	Kernel Density Estimation plot
area	Area plot
pie	Pie plot
scatter	Scatter plot
hexbin	Hexbin plot

The DataFrame plots (cont'd.)

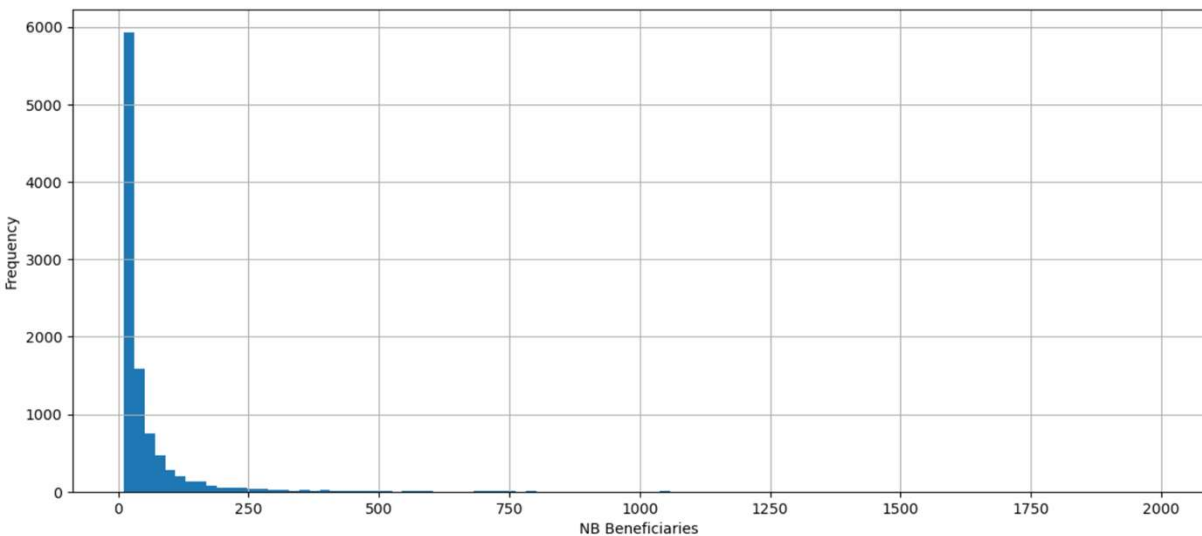
- Plots require different preparations based on what you would like to show.
- Most plots require numerical data for one or both axes, while categorical data can be also used for axes in some plots.
- Some take information on one axis and infer the second.
 - Example: Histograms, pdf, etc.

```
spending_by_specialty_df = pd.read_csv('data/spending_by_specialty.csv', index_col='specialty')
spending_by_specialty_df.head()
```

	spending
specialty	
ADDICTION MEDICINE	920.06
ALLERGY/IMMUNOLOGY	189174.06
ANESTHESIOLOGY	142804.73
CARDIAC ELECTROPHYSIOLOGY	225543.62
CARDIAC SURGERY	12432.92



```
ax = spending_df["nb_beneficiaries"].plot(kind='hist', bins=100, figsize=(14,6), grid=True)
ax.set_xlabel("NB Beneficiaries")
ax.set_ylabel("Frequency");
```



Plot examples

- Pandas makes it easy to create simple and attractive plots.
 - For more examples, check out the [Chart Visualization](#) section in the Pandas documentation.
- For greater flexibility and customization, we will introduce and primarily use the [Matplotlib](#) library.

Exercise: Data exploration and basic visualization with Pandas

- The file `exercise_eda.ipynb` is available on Lamaku. Please download it and attempt to complete the exercises. If you have any questions, feel free to post them on Slack or contact us directly.
- Solutions will be posted at a later time.